Gestion de plusieurs types de diagnostics



Rapport du 3ème projet

Licence professionnelle développeur informatique

Promotion 2023/2024 UHA 4.02

Jean-Luc VASSAL

Remerciements

Je tiens à remercier tous ceux sans qui ce projet n'aurait pas eu les mêmes résultats.

Tout d'abord Soléa qui me permet de venir tous les jours en diminuant mon empreinte carbone.

M&Ms qui apporte une touche de couleur et une saveur particulière à ces moments de travail.

Airbourne, AC/DC, Joe Satriani, Volbeat, Supertramp, Bryan Adams, David Lee Roth, Genesis, Huey Lewis & the news, Joe Bonamassa, Marillion, Pat Benatar, Yes et Sabaton, pour l'énergie qu'ils m'ont transmis lorsque mes batteries étaient à plat.

Ma moitié pour son support, son écoute et aussi ses merveilleux petits plats.

Sans oublier Cimba, fidèle petit compagnon canidé à l'énergie débordante et communicative.

Sommaire

Introduction 1
Les Tisserands d'EBN 2
La Cité du Réemploi 2
La demande initiale du client 2
L'équipe projet 3
Le déroulement du projet vélo 3
    La situation de départ et le cahier des charges 3
    Nos choix de technologies 4
    Structure de notre application 4
    Implémentation des types de diagnostics 5
    Les difficultés techniques 7
    Le résultat obtenu 8
    Que reste-t-il à faire ? 8
Conclusion 9

Introduction

La formation à l'UHA 4.0 suit une approche pédagogique différente et novatrice, qui se traduit entre autres par plusieurs projets basés sur des demandes d'entreprises.

Dans le cadre du 3ème projet de ma formation, je fais partie du groupe "projet vélo".

Ce projet a pour objectif de digitaliser l'enregistrement de diagnostics fait sur des vélos par l'association les tisserands d'EBN au sein de la cité du réemploi de Sausheim.

Il est réalisé par une équipe de 6 étudiants, utilise le langage PHP et le Framework Symfony.

La gestion de plusieurs types de diagnostics étant ma contribution majeure à ce projet, elle sera plus particulièrement développée dans les pages suivantes.

Nous aborderons successivement dans ce rapport : le porteur de projet Gilles ANNENKOFF pour les Tisserand d'EBN, la demande du client, la situation actuelle, la réflexion initiale, la réalisation du projet et une conclusion.

Les Tisserands d'EBN

Association locale existant depuis près de 10 ans, elle est composée d'environ une cinquantaine de membres. Elle mène des actions variées sur le thème du bien vivre ensemble dont :

- La création et l'animation d'un jardin partagé à l'arrière de la Filature
- La Création de deux haies fruitières en plein centre-ville de Mulhouse
- La mise en place d'une boîte à livres
- La création et la gestion d'un site de compostage partagé

Elle intervient également sur le thème des mobilités douces sur Mulhouse et la M2a.

Dans ce cadre elle récupère et répare des vélos en partenariat avec la Cité du Réemploi pour permettre la réutilisation de ces vélos par divers publics.

Elle réalise aussi, en partenariat avec les collectivités, la ville de Mulhouse et la MACIF des diagnostics vélo auprès d'écoles, d'institutions, de collectivités et de sociétés.

Elle est cofondatrice de la future Cité du Vélo de Mulhouse.

La Cité du Réemploi

Située à Sausheim /Illzach-Modenheim, elle est ouverte depuis le 20 novembre 2021.

C'est une association qui regroupe dans un seul bâtiment, une recyclerie et des magasins spécialisés "réutilisation" (meubles, électroménager et textile). Elle organise également des ateliers participatifs sur différents thèmes (bois, petit bricolage, couture et vélo).

Elle héberge actuellement l'atelier de diagnostics et de réparations de vélos géré par les tisserands d'EBN.

La demande initiale du client

Dans le cadre de son activité Vélo, les Tisserands d'EBN reçoivent et gèrent au sein de la cité du réemploi plusieurs centaines de vélos. Ils doivent également intervenir dans des manifestations sur d'autres sites où ils réalisent des diagnostics avec un atelier mobile. Ce travail se fait aujourd'hui uniquement sur papier.

C'est dans ce cadre que l'association s'adresse à l'UHA 4.0. La gestion sous forme papier pose des problèmes en termes de traçabilité et de suivi et n'est plus adaptée au volume traité annuellement. Elle souhaite donc remplacer son traitement papier par une application numérique.

L’équipe projet

Notre groupe est composé de 6 étudiants, tous en 2ème année.

- Najib EBRAHIMI
- Medhi GIRARD
- Adam MADOUX (Scrum master)
- Liz MANRESA
- Adam MOUKHLISS
- Jean-Luc VASSAL

Cette équipe est hétérogène en termes de compétences. Nous travaillons suivant la méthode AGILE et le Framework SCRUM et nous utilisons JIRA et GIT pour structurer notre travail.

Elle est suivie par Pierre SCHULLER comme tuteur pédagogique.

Le déroulement du projet vélo

La situation de départ et le cahier des charges

Le client a remis un cahier des charges succinct. Ce dernier explique brièvement les étapes mises en place (identification du vélo, diagnostic, réparation ou récupération de pièces et destruction). Le résultat du diagnostic doit permettre une traçabilité des actions menées et être consultable par le propriétaire du vélo.

Le cahier des charges et les échanges avec le client mettent en lumière la difficulté de traitement et d’utilisation uniforme par tous du document papier qui contient 47 éléments de contrôle.

De plus cela précise certaines attentes :

- Un fonctionnement orienté tablette pour être mobile
- Un coût d’exploitation le moins cher possible
- La possibilité de travailler à plusieurs utilisateurs en même temps
- Une sauvegarde des informations sur serveur
- La possibilité d’effectuer des recherches multicritères
- Une utilisation et une ergonomie simple et fluide
- Une sécurisation des saisies, pour assurer une phase de renseignement respectant le même format quel que soit l’utilisateur

Rapidement la demande de reprise du format papier d’un seul type de diagnostic avec la totalité des 47 éléments de contrôle évoluera vers un besoin de plus de souplesse. Celui-ci se traduira par la mise en place d’une solution permettant d’avoir plusieurs types de diagnostics contenant seulement une partie des éléments de contrôle.

Nos choix de technologies

Afin de répondre à ce cahier des charges, nous décidons de réaliser cette application avec le langage PHP, le Framework Symfony et PHPMyAdmin comme base de données, le tout dans un docker.

L'application étant majoritairement composée de formulaires de saisie et d'affichages d'état, l'outil Symfony très efficace dans ce domaine s'avère un choix judicieux. Un avantage supplémentaire étant aussi son moindre coût d'hébergement.

Structure de notre application

Nous commençons par une phase de réflexion en groupe afin de définir les éléments suivants :

Le MVP (minimum viable product en anglais) qui correspond à une version permettant de valider au minimum le besoin client. Il contient la possibilité d'enregistrer et de consulter un vélo, la possibilité de réaliser un diagnostic identique au contenu du document papier et la possibilité de reprendre un diagnostic non terminé.

Cela nous permet de définir globalement la structure de notre application (cf. figure 1)

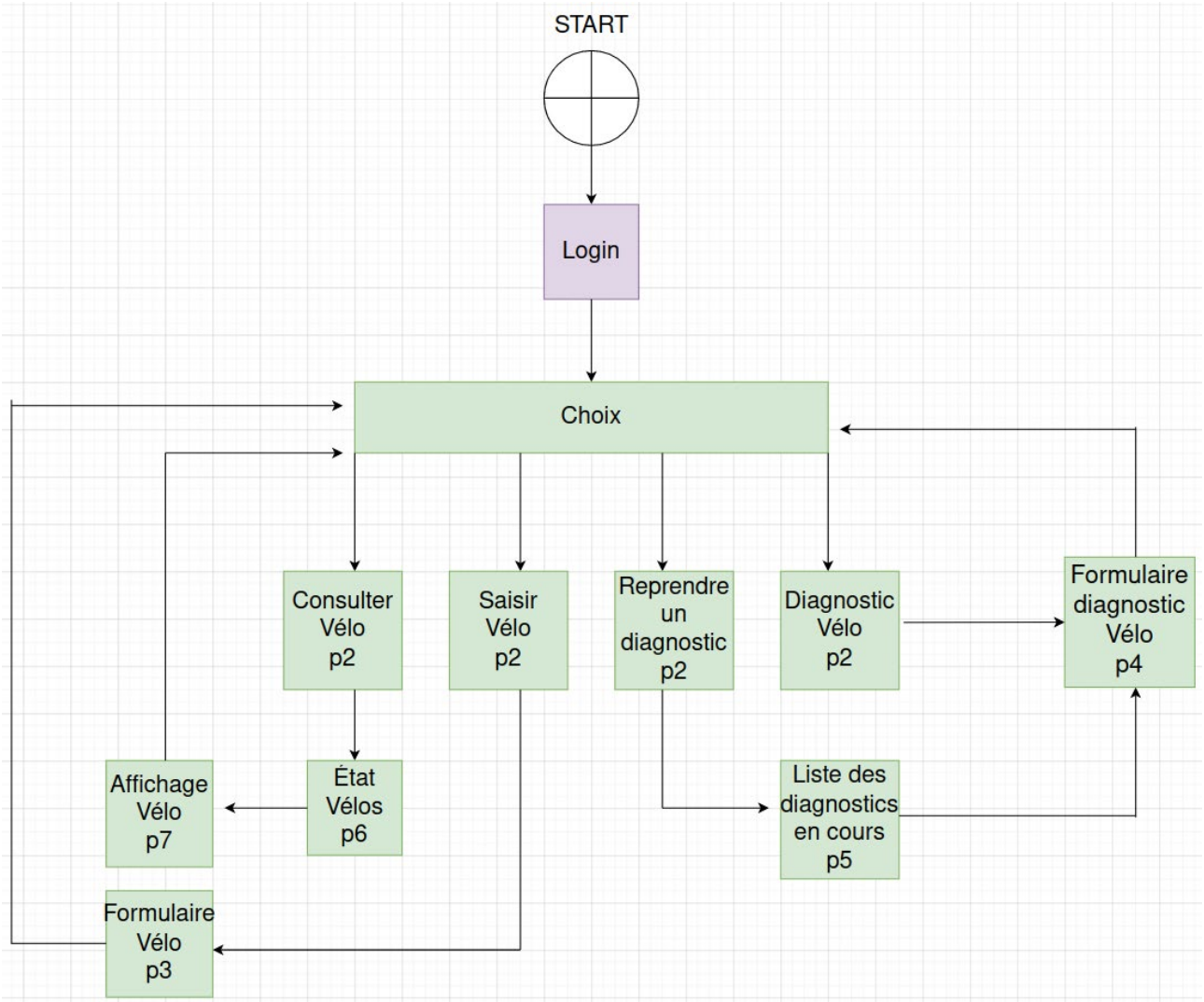


Figure 1, structure initiale
p(x) est le numéro de la page

Nous réalisons également les maquettes de l'application afin d'avoir un fil conducteur pour notre conception. Mon rôle est de travailler principalement sur l'aspect diagnostic de l'application.

Implémentation des types de diagnostics

Le schéma de la base de données a d'abord correspondu à la demande initiale du client (cf. Figure 2) consistant à reproduire le diagnostic des 47 éléments de contrôle.

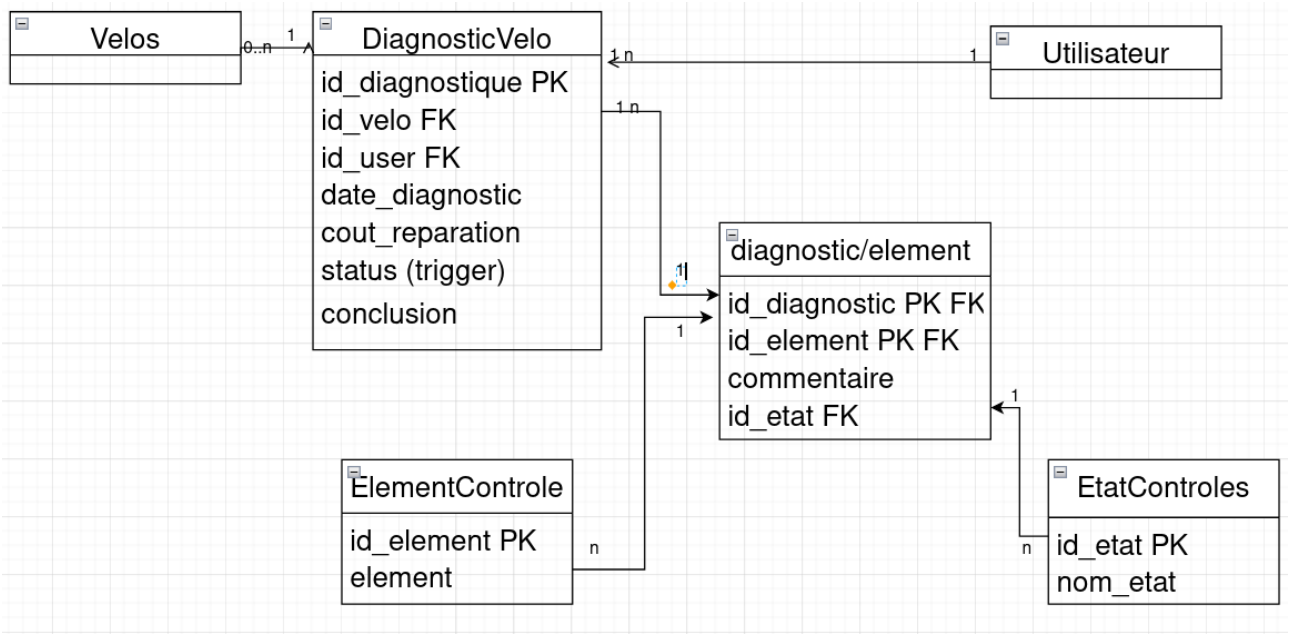


Figure 2. 1ère version du schéma de la base de données

Dans cette structure les 47 éléments de contrôle sont des valeurs enregistrées dans la table « *ElementControle* ». Cette table est liée à la table « *DiagnosticVelo* » qui identifie le diagnostic par la table de liaison « *Diagnosticelement* ». C'est dans cette dernière que sera, pour chaque diagnostic, enregistré l'ensemble des 47 éléments de contrôle ainsi que les commentaires. Cette table est en relation avec la table « *Etatcontroles* » afin de pouvoir enregistrer pour chaque élément son état (OK, à réparer, Pas OK, N.A).

Il faut également mettre en place une logique pour définir automatiquement le statut en cours ou terminé d'un diagnostic. Ce statut est important car il permet de retrouver et de reprendre facilement les diagnostics non-terminés. Mais il doit être automatique et non dépendant de l'utilisateur pour éviter des oublis. Cette logique sera mise en place via un « déclencheur » (trigger en anglais) en SQL directement dans la base de données. Ce choix permet ensuite de facilement récupérer cette information dans les différents états.

Après avoir montré au client cette version, il a fallu la faire évoluer pour avoir plus de modularité dans les diagnostics au lieu d'avoir un seul type de diagnostic possible.

Suite à une série de changements, la dernière version (cf. Figure 3) permet la gestion de différents types de diagnostics. Cette approche répond mieux au besoin d'adapter le diagnostic réalisé en fonction du lieu ou du type d'action menées.

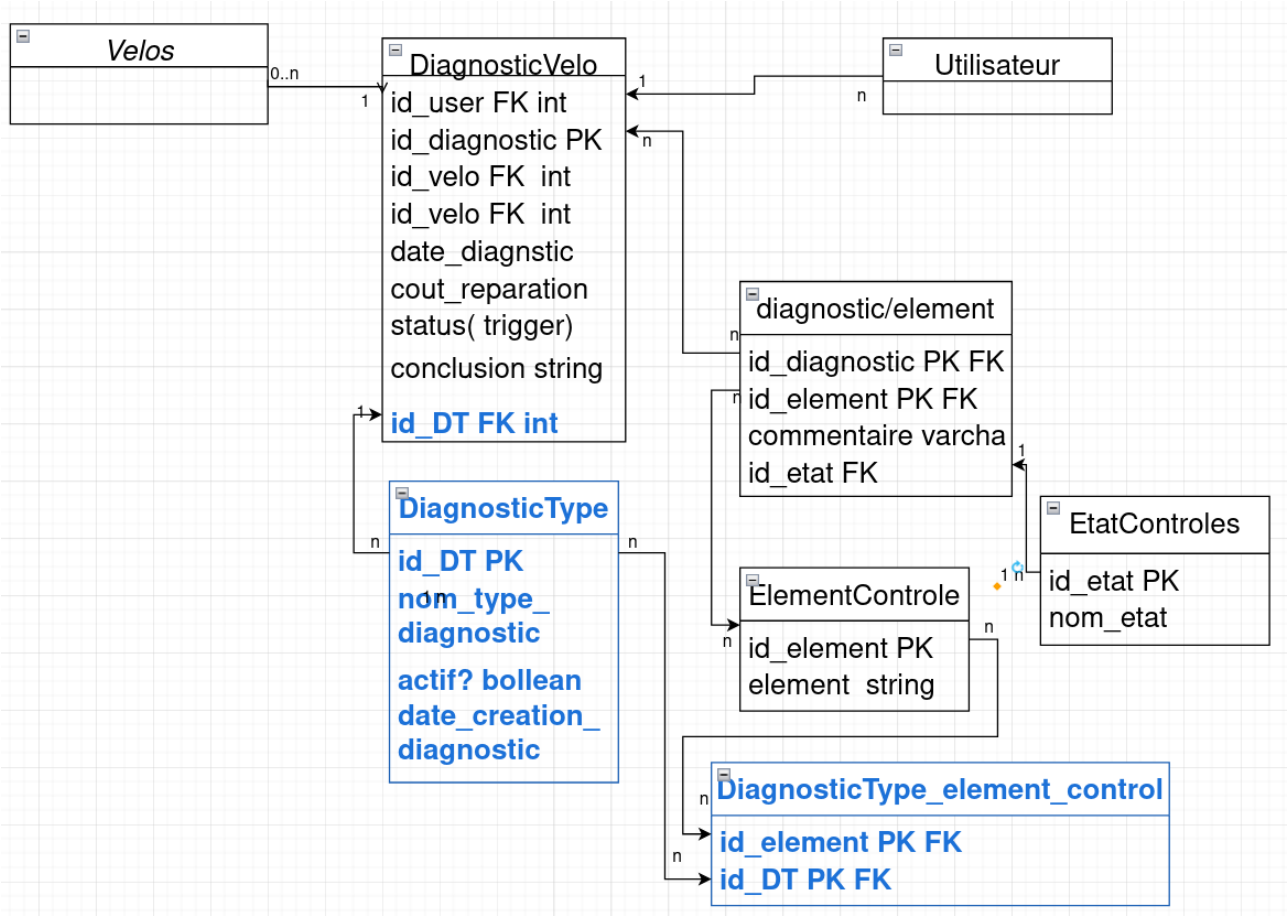


Figure 3. V4 du schéma de la base de données

Cette évolution du fonctionnement de la base de données doit permettre de gérer plusieurs types de diagnostics et également de limiter l'impact sur le code existant.

L'objectif est de permettre d'avoir plusieurs types de diagnostics contenant un nombre différent d'éléments de contrôle. Mais également de permettre au client d'avoir la possibilité de créer, de façon autonome, les types correspondant à son besoin.

Cela nécessite de ma part plusieurs modifications.

La création de 2 nouvelles tables, « *DiagnosticType* » et « *DiagnosticType_element_control* ». De nouvelles liaisons sont mises en place entre ces tables et les tables existantes. Cela se fait grâce à l'ORM Doctrine de Symfony.

La table « *DiagnosticType* » permet d'enregistrer un type de diagnostic avec un nom. La table «*DiagnosticType_element_control* » permet d'associer les éléments de contrôle de son choix à un type de diagnostic.

Le même élément de contrôle peut donc être utilisé dans plusieurs types de diagnostics. Un type de diagnostic peut contenir de un à la totalité des éléments de contrôle existant.

Il faut également modifier le « déclencheur » qui permet de définir dynamiquement le statut du diagnostic, terminé ou en cours.

La logique de l'application lors de la création de diagnostics doit aussi être remaniée. Il y a maintenant une nouvelle étape avec une page présentant les types de diagnostics possibles. Cette page s'adapte dynamiquement au nombre de types. L'utilisateur pouvant créer des types de diagnostics et les rendre actifs ou non. Seuls les types actifs s'afficheront.

Je dois également reprendre le contenu du « *diagnosticController* » présent dans Symfony. Ceci afin que, lors de la création d'un diagnostic, ce dernier corresponde à un type. Ce qui a un impact sur les éléments de contrôle affichés. Il faut aussi modifier la structure du code gérant la reprise d'un diagnostic en cours. Le fonctionnement est maintenant capable de récupérer un diagnostic en lui renvoyant uniquement les éléments, déjà remplis ou vides, correspondant au type. L'enregistrement des modifications fonctionne comme avant.

Les difficultés techniques

Dans la 1ère version, le "déclencheur" permet automatiquement de définir le statut terminé ou en cours du diagnostic, en comptant le nombre d'éléments renseignés par rapport aux 47 présents. Mais ce nombre fixe de 47 ne permet pas d'évolution de l'application en cas de création de nouveaux éléments de contrôles.

Il faut donc transformer la logique de la requête pour compter automatiquement le nombre d'éléments attendus. De plus avec une approche basée sur plusieurs types de diagnostics, le nombre d'éléments attendus est variable. Il est nécessaire d'arriver à comparer le nombre d'éléments présent dans un type de diagnostic avec celui enregistré dans un diagnostic d'un type. Le "déclencheur" est maintenant capable de façon dynamique de s'adapter à la différence du nombre d'éléments attendus suivant le type de diagnostic.

La structure MVC (Modèle, Vue, Controller), liée à la structure orientée objet de Symfony, m'a posé quelques problèmes.

Lors de la modification du code pour la reprise d'un diagnostic en cours, la récupération des informations depuis 4 tables différentes s'avère difficile. Les noms des variables se ressemblent et sont source de confusion de ma part. L'information, qui doit être récupéré via une requête avec l'ORM Doctrine de Symfony, doit être transmise à plusieurs fichiers. J'utilise la commande dd() pour afficher le contenu des variables. Mais identifier les informations récupérées et leur cheminement pour s'afficher correctement dans la vue me prend plusieurs jours et passe par le changement de nom d'une variable.

Le résultat obtenu

A l'issue des 6 semaines de ce projet qui est parti de zéro, nous livrons au client une application répondant à 5 grandes fonctionnalités.

L'enregistrement et la modification d'un vélo est opérationnelle. La gestion de plusieurs utilisateurs et profils d'utilisateurs fonctionne à travers les rôles utilisateur et administrateur et la possibilité d'enregistrer de nouveaux utilisateurs.

La gestion de diagnostic, qui est le cœur de ce projet, permet un fonctionnement totalement paramétrable. Il est possible de créer des éléments de contrôles et des types de diagnostics les contenant tous ou en partie. Les types peuvent être rendus actifs ou inactifs.

On peut créer et reprendre, s'il n'est pas terminé, un diagnostic correspondant à un type. L'application permet de retrouver immédiatement les diagnostics en cours afin de pouvoir les terminer.

La consultation des vélos comme des diagnostics est possible. Les tableaux récapitulatifs peuvent être filtrés sur plusieurs critères pour faciliter la recherche.

Enfin l'impression d'un diagnostic, qu'il soit terminé ou en cours, est possible sur au maximum un A4 recto-verso.

Que reste-t-il à faire ?

Certains besoins sont apparus au cours du développement de l'application.

La durée de 6 semaines du projet n'a pas permis de les concrétiser.

Il s'agit :

- De la possibilité pour l'administrateur d'ajouter des marques de vélo à la liste déroulante.
- De la possibilité d'extraire les informations vers un fichier PDF ou Excel.
- D'avoir des statistiques concernant les diagnostics, leurs résultats et le type de public visé.
- De pouvoir enregistrer une réparation liée à un diagnostic.
- De la possibilité d'imprimer une étiquette avec le numéro du vélo lors de son enregistrement.
- De la gestion du prêt ou de la vente des vélos, utile à la structure mais qui n'était pas prévue dans le cadre de ce projet.

Conclusion

Ce projet qui a connu des moments délicats ou inattendus, comme une présentation devant Madame la ministre de l'enseignement supérieur, a abouti à un résultat de qualité.
Nous avons réussi, non seulement à répondre à la demande du client, mais également à prendre en compte de nombreux ajouts.
L'application que nous livrons dépasse le cahier des charges et est déployée chez le client. Il a déjà réalisé des tests au sein de sa structure.

A mon niveau, j'ai particulièrement aimé la problématique proposée par ce projet ainsi que les rapports avec le client.
J'ai pu mettre en œuvre mes connaissances acquises en gestion de projet et l'utilisation de JIRA en particulier. J'ai également pu approfondir ma connaissance du Framework Symfony et m'appuyer sur les apports de ma 1ere année en PHP, HTML/CSS et un peu en Javascript. J'ai aussi pu utiliser les apports théoriques en UML et SQL vus en 2ème année.

Ce projet permet d'ores et déjà au client de ne plus utiliser le document papier qu'il avait pour but de remplacer. Une 2ème itération devrait permettre d'ajouter certaines fonctionnalités comme la gestion des réparations, le suivi statistique et l'export de données.

UHA 4.0 Projet n°3

Année 2023/2024

Création d’une application web pour réaliser et consulter différents types de diagnostics de vélos.

Mots clefs :

- PHP
- Symfony
- MySQL
- Base de données
- Vélos
- Diagnostics